SOFTWARE ENGINEERING LAB

LAB 7

Fuzzing for Input Validation Bugs

Experiment Report

Course	Software Engineering Laboratory
Experiment	Lab 7 – Fuzzing for Input Validation Bugs
Application	Weather Alert input-processing functions
Language	Python
Testing framework	pytest with Hypothesis
Date	12 November 2026

Submitted as documentation of the fuzz-testing experiment, discovered input-validation defects, corrective changes, and final verification.

1. Aim and Objectives

The aim of this experiment is to apply fuzz testing as a dynamic software-testing technique to discover input-validation defects in a Python application, analyse the failures, and improve robustness through defensive programming.

Objectives

• Understand fuzz testing and property-based testing.
• Use Hypothesis to generate unexpected and malformed inputs automatically.
• Identify crashes and incorrect results caused by edge cases.
• Correct the application's input validation without weakening the tests.
• Re-run the tests and verify normal application behaviour.

2. Application Under Test

The assigned Weather Alert application contains three input-processing functions:

Function	Purpose
`parse_reading(text)`	Parses a temperature/humidity reading in temperature/humidity format.
`wind_category(speed_text)`	Classifies wind speed into none, advisory, or warning categories.
`rainfall_spread(readings_csv)`	Parses comma-separated rainfall values and returns the difference between the maximum and mini

3. Tools and Test Method

Python 3.13.2 was used to execute the application. **pytest 9.1.1** ran the test suite, and **Hypothesis 6.170.15** generated property-based test inputs. The tests used strategies such as arbitrary text (including None), integers, and lists of floating-point values including NaN and infinity.

A property-based test describes a rule that should hold for many inputs. Hypothesis generates examples and shrinks failures to small counterexamples that make the defect easier to understand.

The workflow was: run the original application; execute the fuzz tests against the original implementation; inspect the counterexamples; harden the application; then rerun the same tests and normal sample inputs.

4. Initial Application Execution

The original application was first executed with representative valid inputs. The baseline run demonstrates the expected behaviour for a temperature/humidity reading, a wind speed, and a rainfall CSV input.

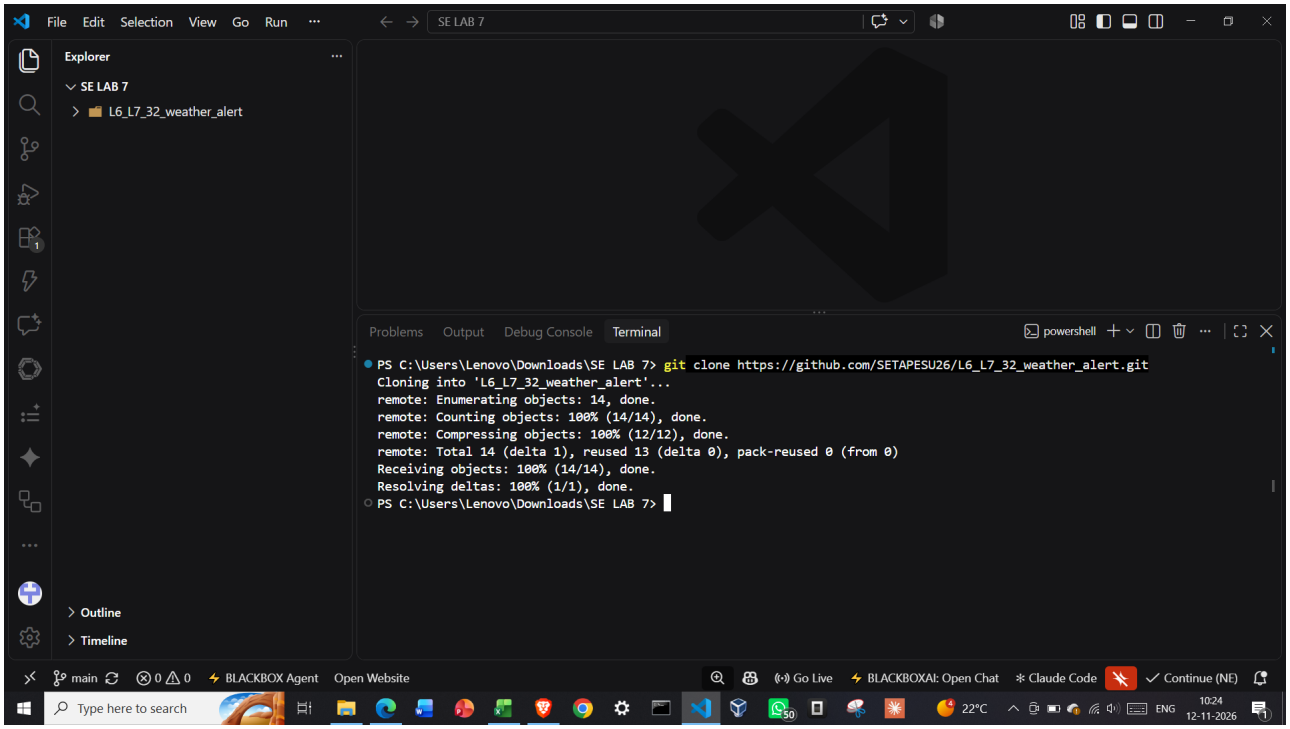


Figure 1. Initial application run with valid sample inputs.

5. Fuzz Testing Before Fixes

The initial Hypothesis test run collected six tests. All six failed against the original implementation. These failures were retained as evidence of defects; the test expectations were not changed to make the suite pass.

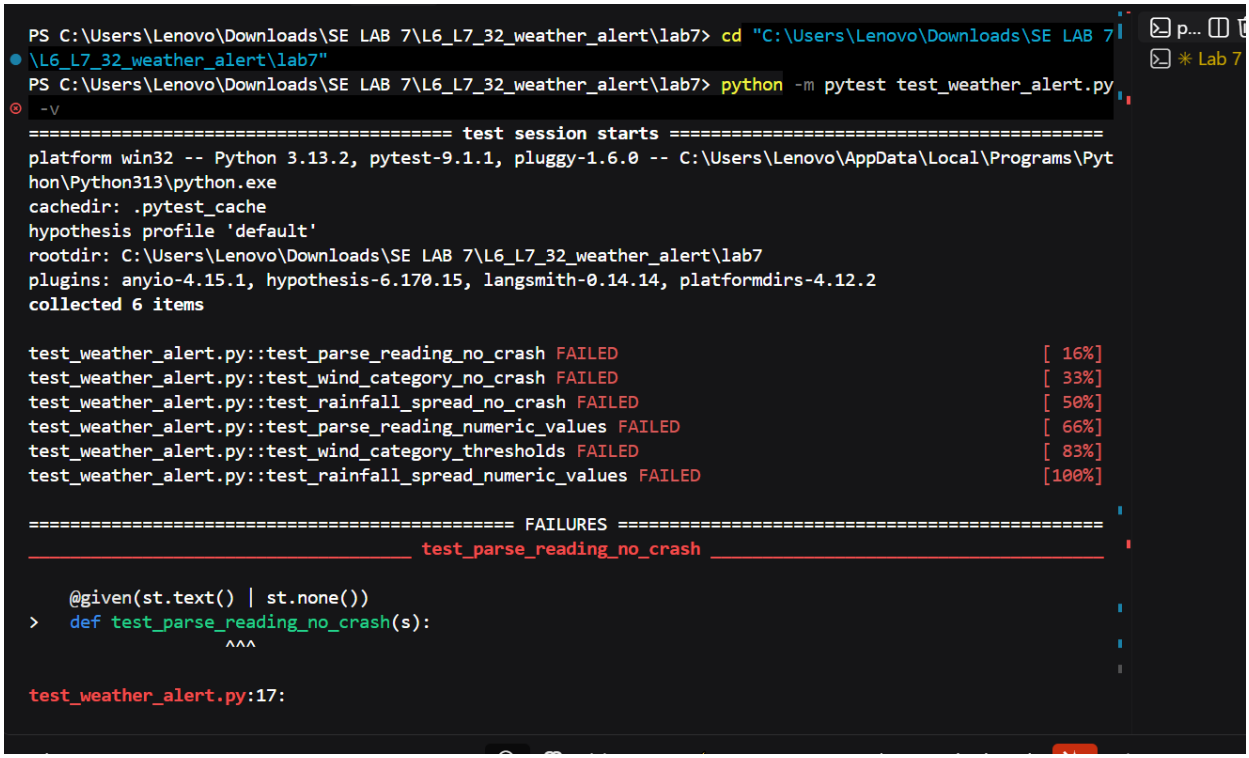


Figure 2. Initial pytest run showing all six property-based tests failing.

6. Defects Identified

Hypothesis reduced several failures to small counterexamples. The main issues observed in the failing output were:

Area	Counterexample / observed behaviour	Defect
parse_reading	None caused AttributeError when split() was called.	Input type was not validated before string operations.
parse_reading	'inf/0' was accepted; '0.0/-1' accepted invalid humidity.	Non-finite temperature and out-of-range humidity were not rejected.
wind_category	None caused TypeError; -1 returned 'none'.	Input type and non-negative speed were not validated.
rainfall_spread	None caused AttributeError; '-1.0' returned 0.0.	Input type and non-negative rainfall values were not validated.

The initial test output also showed an ExceptionGroup for the numeric temperature/humidity property test because Hypothesis found more than one distinct invalid acceptance (infinite temperature and negative humidity).

7. Corrective Changes

The application was hardened while retaining the six property-based tests. The reported changes included:

• Added input-type and empty-input validation so malformed values are rejected with ValueError rather than causing unexpected TypeError or AttributeError exceptions.
• Validated the temperature/humidity format and required exactly one separator.
• Rejected non-finite temperature and rainfall values using finite-number checks.
• Restricted humidity to the inclusive range 0–100.
• Rejected negative wind speeds and negative rainfall readings.
• Added cleaning for surrounding whitespace and invisible Unicode formatting/filler characters encountered during input handling.

The fixes were implemented in `weather_alert.py`. The property-based tests remained in `test_weather_alert.py` and were not weakened.

8. Application Error Example

```
PS C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7> python weather_alert.py
==== Running parse_reading ====
Enter temperature/humidity (e.g. 35.5/80): 35.5
Traceback (most recent call last):
  File "C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7\weather_alert.py", line 80, in <mo
dule>
    main()
    ~~~~^^
  File "C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7\weather_alert.py", line 66, in mai
n
    result = parse_reading(user_input)
  File "C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7\weather_alert.py", line 18, in par
se_reading
    raise ValueError(f"reading must be 'temperature/humidity', got {text!r}")
ValueError: reading must be 'temperature/humidity', got '35.5'
==== Running parse_reading ====
Enter temperature/humidity (e.g. 35.5/80): (35.5/80)
Traceback (most recent call last):
  File "C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7\weather_alert.py", line 80, in <mo
dule>
    main()
    ~~~~^^
  File "C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7\weather_alert.py", line 66, in mai
n
    result = parse_reading(user_input)
  File "C:\Users\Lenovo\Downloads\SE LAB 7\L6_L7_32_weather_alert\lab7\weather_alert.py", line 24, in par
```

Figure 3. Example of the original parser raising ValueError when the input did not match the required temperature/humidity format.

9. Final Verification

After the validation changes, the same six tests were executed again. The reported final run completed with all six tests passing.

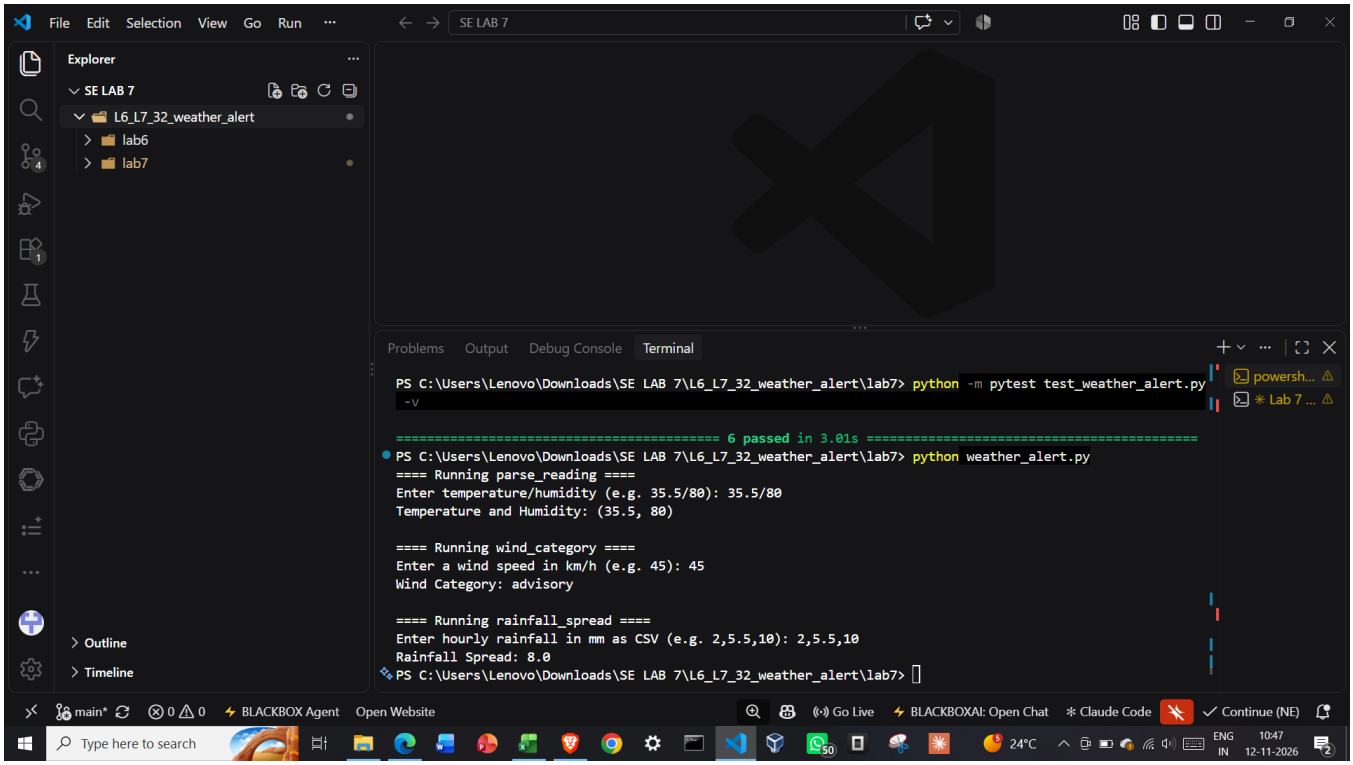


Figure 4. Final verification: six tests passed and the application produced the expected sample outputs.

The normal sample run used the following inputs and produced the corresponding results:

Input	Expected / observed output
Temperature/humidity: 35.5/80	(35.5, 80)
Wind speed: 45 km/h	advisory
Rainfall CSV: 2,5.5,10	8.0

10. Conclusion

This experiment demonstrated how property-based fuzz testing can reveal defects that ordinary sample inputs may not expose. Hypothesis generated unexpected inputs and reduced failures to useful counterexamples, including None, infinite values, invalid humidity, negative wind speed, and negative rainfall. Defensive input validation was then added to the application. The final test run passed all six tests, and the normal application inputs continued to produce the expected results.

The experiment met the handout's core objective: identify input-validation bugs through fuzzing, correct the application, and verify the changes with the same test suite.

Files associated with the experiment

- weather_alert.py – fixed application
- test_weather_alert.py – Hypothesis property-based tests
- README.md – experiment/project notes

Reference: Lab 7 – Fuzzing for Input Validation Bugs, student handout supplied for the experiment.